Informe de monitoreo de una API FastAPI con New Relic

Buscador eficiente de documentos

Trabajo práctico para presentación en clase
Evidencias obtenidas durante las pruebas del 14 de octubre de 2026

Aplicación observada: Buscador Eficiente - PEF
Tecnologías: Python, FastAPI, Uvicorn y agente New Relic Python
Endpoint principal: GET /buscar

1. Resumen ejecutivo

Se instrumentó la API del buscador de documentos con el agente New Relic para observar su comportamiento mientras recibía solicitudes HTTP reales. La aplicación se inició mediante **newrelic-admin.exe run-program**, se configuró una License Key válida y se registró en APM con el nombre **Buscador Eficiente - PEF**.

Las capturas muestran que New Relic recibió solicitudes del endpoint **/buscar**, registró transacciones web, calculó tiempos de respuesta, contabilizó el throughput, generó trazas distribuidas y almacenó logs. La respuesta funcional de la API fue HTTP 200 y una consulta de ejemplo devolvió 957 documentos para el término **algoritmo**.

Conclusión principal: la integración con New Relic quedó operativa y permitió analizar el rendimiento de la API desde el punto de vista de una aplicación en ejecución.

2. Objetivos

• Instrumentar la API FastAPI sin modificar su lógica de búsqueda.
• Registrar solicitudes HTTP y medir sus tiempos de respuesta.
• Analizar throughput, errores, transacciones, trazas y logs.
• Relacionar las métricas observadas con el funcionamiento del buscador.
• Obtener evidencias visuales para explicar el trabajo en clase.

3. Configuración y metodología

La aplicación se ejecutó desde el entorno virtual del proyecto con Python 3.13, Uvicorn y el agente New Relic Python 14.0.0. El servidor quedó disponible en **http://127.0.0.1:8002**. La ejecución instrumentada tuvo esta estructura:

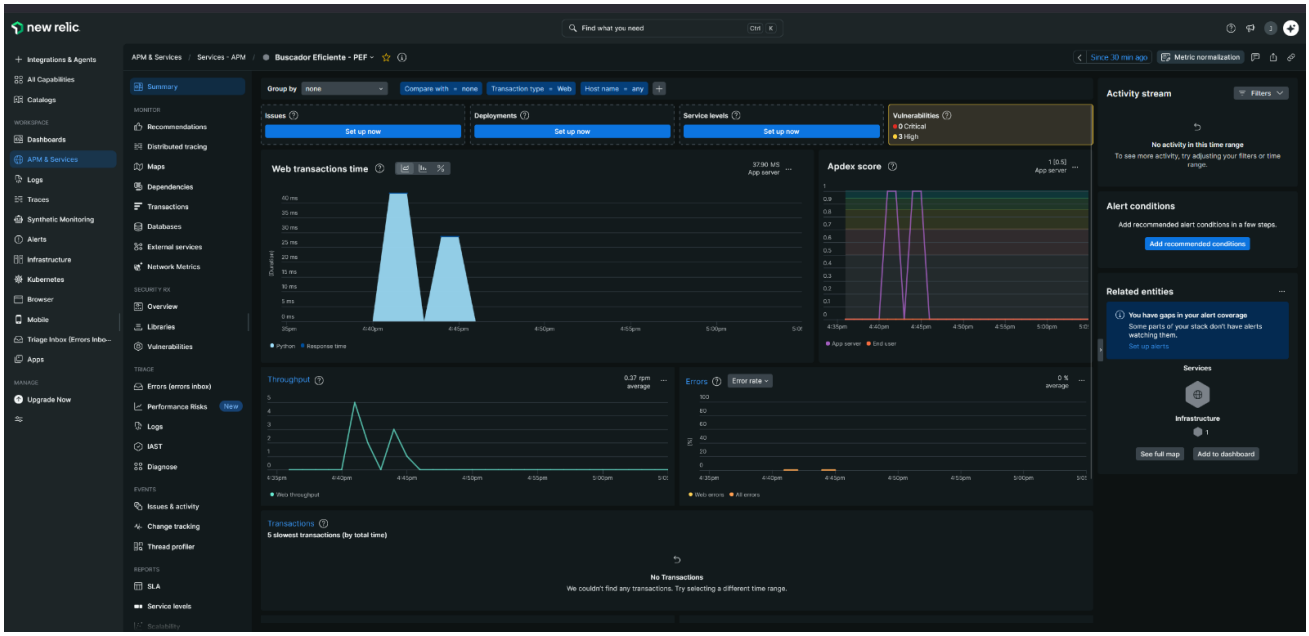
```
newrelic-admin.exe run-program python.exe -m uvicorn api:app --app-dir src
--port 8002
```

Luego se realizaron consultas al endpoint con términos como **algoritmo**, **python** y **archivo**. Se conservaron las capturas que aportaban información medible y se descartaron duplicados y una vista sin actividad suficiente.

4. Traducción de los principales indicadores

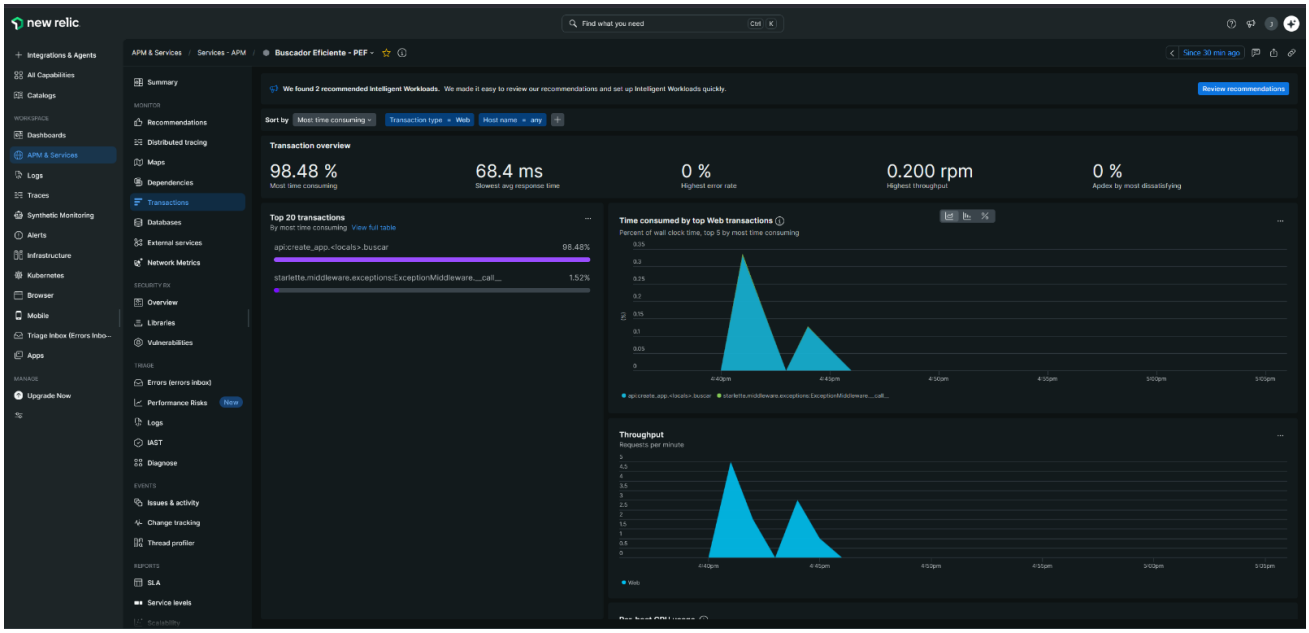
Indicador original	Traducción	Qué representa
Summary	Resumen	Vista general del servicio.
Web transactions time	Tiempo de transacciones web	Duración de las solicitudes HTTP.
Throughput	Rendimiento / solicitudes por minuto	Cantidad de solicitudes recibidas.
Error rate	Tasa de errores	Porcentaje de solicitudes con error.
Apdex score	Índice Apdex	Satisfacción del usuario según el tiempo de respuesta.
Transactions	Transacciones	Operaciones web agrupadas por nombre.
Distributed tracing	Trazas distribuidas	Recorrido interno de una solicitud.
Logs	Registros	Mensajes emitidos por la aplicación o el servidor.
Response time	Tiempo de respuesta	Tiempo empleado para atender una operación.

5. Evidencia 1: resumen de rendimiento



Interpretación: el gráfico de tiempo de transacción muestra picos cercanos a 41 ms y 27 ms. El throughput presenta varias solicitudes alrededor de las 16:40 y 16:45. El error rate queda prácticamente en cero durante el intervalo observado. El Apdex alcanza valores cercanos a 0,9, lo que indica una experiencia satisfactoria para las solicitudes registradas.

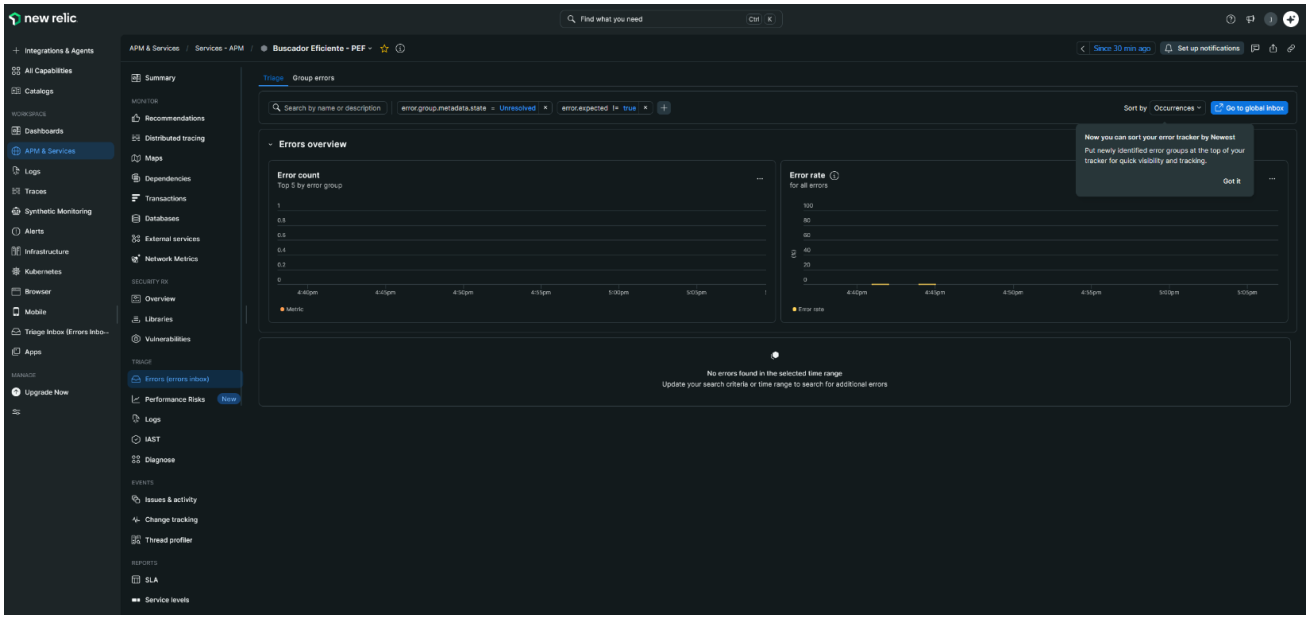
6. Evidencia 2: transacciones web



Datos visibles: la transacción principal `api:create_app.<locals>.buscar` concentra el 98,48 % del tiempo consumido. La transacción del middleware de excepciones representa el 1,52 % restante. El tiempo de respuesta promedio más lento observado es de 68,4 ms; el throughput máximo registrado es 0,200 rpm y la tasa máxima de error indicada es 0 %.

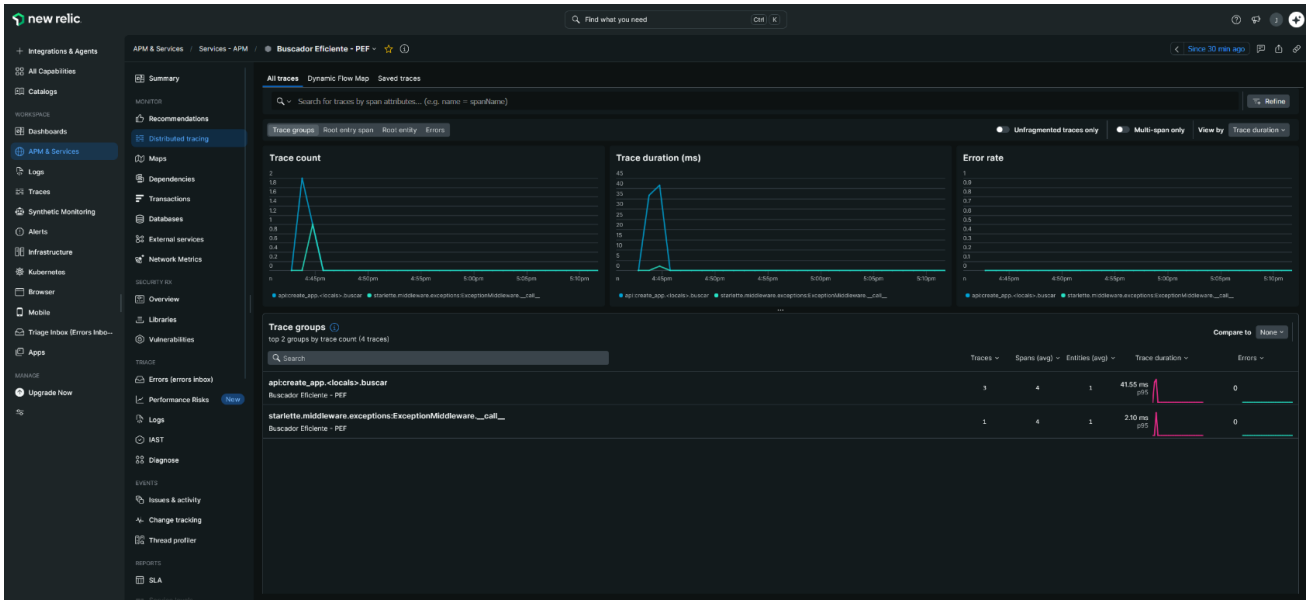
Interpretación: el procesamiento está concentrado en el handler **buscar**, que es exactamente el punto funcional que interesa analizar. La presencia del middleware no implica un fallo: forma parte de la cadena normal de atención de la solicitud.

7. Evidencia 3: errores



La vista de errores no encuentra errores aplicacionales dentro del rango seleccionado. El pequeño registro de tasa de errores observado en otras capturas coincide con solicitudes a rutas no definidas, como **/** o **/favicon.ico**, que responden 404. No debe confundirse un 404 de una ruta auxiliar con un fallo del endpoint principal **/buscar**, que respondió 200 durante las pruebas.

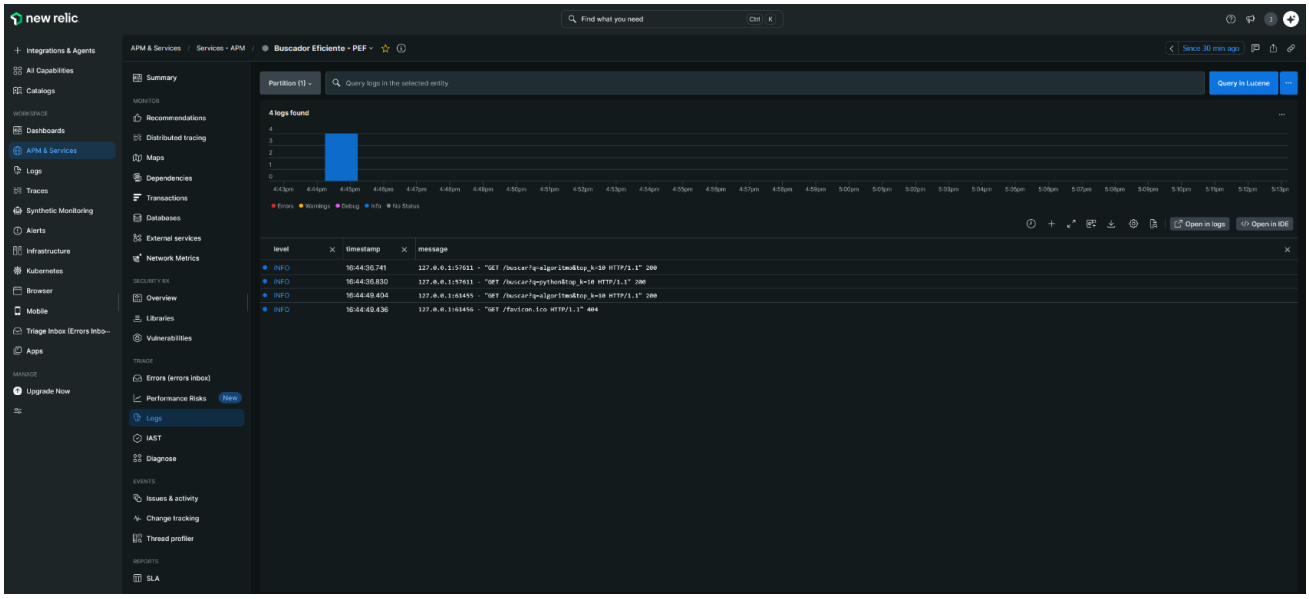
8. Evidencia 4: trazas distribuidas



New Relic registró cuatro trazas en el período mostrado. La traza principal corresponde a la transacción del buscador y presenta una duración de hasta aproximadamente 41,55 ms en el percentil 95 mostrado. También aparece la traza del middleware de excepciones, con una duración aproximada de 2,10 ms en su percentil indicado. El error rate de las trazas es 0.

Utilidad: una métrica indica cuánto tarda una operación, mientras que una traza ayuda a entender por dónde pasó la solicitud. En este caso, la traza confirma que la mayor parte del tiempo está asociada al handler del buscador y no al middleware.

9. Evidencia 5: logs de las solicitudes



Los registros muestran cuatro solicitudes concretas: tres consultas válidas al endpoint **/buscar** con respuestas HTTP 200 y una solicitud a **/favicon.ico** con respuesta 404. Las consultas registradas fueron **algoritmo**, **python** y **archivo**. Esta captura es una de las evidencias más importantes porque relaciona las métricas de New Relic con acciones concretas realizadas sobre la API.

10. Análisis técnico del comportamiento

La primera consulta puede presentar un costo mayor porque la API construye o actualiza el índice del corpus antes de responder. Una vez que el índice queda disponible, las consultas siguientes reutilizan esa estructura y evitan repetir todo el trabajo de lectura e indexación. Por ese motivo, es esperable observar una primera medición más alta y luego tiempos menores.

Las capturas de New Relic complementan el análisis de Scalene. Scalene permite estudiar el costo de líneas de código y diferenciar CPU Python, CPU nativo y memoria. New Relic observa el servicio funcionando: solicitudes, transacciones, errores, trazas y logs. Usadas juntas, ambas herramientas permiten relacionar el hotspot interno con el efecto visible para el usuario.

11. Limitaciones y observaciones

• La prueba tuvo pocas solicitudes y se realizó en un entorno local, por lo que no representa una carga de producción.
• Los gráficos de New Relic dependen del rango temporal seleccionado y pueden mostrar 0 rpm aunque hayan existido solicitudes aisladas.
• La vista de errores incluye rutas auxiliares no implementadas, como favicon, pero el endpoint funcional respondió correctamente.
• Las capturas fueron tomadas en inglés; este informe traduce los indicadores y conserva los nombres técnicos originales cuando son necesarios para ubicarlos en la interfaz.

12. Conclusiones

La integración con New Relic fue exitosa: el agente Python 14.0.0 aceptó la License Key, registró el servicio **Buscador Eficiente - PEF** y comenzó a reportar datos. El endpoint **/buscar** generó transacciones web, logs con respuestas 200 y trazas sin errores. El tiempo observado fue bajo, con picos de aproximadamente 27 a 41 ms en el resumen y un máximo promedio indicado de 68,4 ms en la vista de transacciones.

El resultado permite defender técnicamente que el sistema funciona y que New Relic es útil para observar el comportamiento real de una API, mientras que Scalene aporta el diagnóstico detallado del código que produce ese comportamiento.

13. Guion breve para la exposición

“Instrumenté mi API FastAPI con el agente New Relic Python y la ejecuté mediante newrelic-admin. Realicé tres búsquedas sobre el endpoint /buscar. New Relic registró las solicitudes como transacciones web, mostró los tiempos de respuesta, el throughput y el Apdex. Las trazas indican que la mayor parte del tiempo se consume en el handler del buscador. Los logs confirman tres respuestas HTTP 200 y un 404 correspondiente al favicon. Por lo tanto, la aplicación funciona correctamente y el monitoreo permite observar su rendimiento desde el punto de vista del usuario.”